

Encapsulation

The Art of Data Protection



Bundling Data and Behavior Securely

HIDE • PROTECT • VALIDATE

1. The Protective Layer

Encapsulation is the OOP principle that bundles data (attributes) and the code that operates on that data (methods) into a single unit called a **Class**.

Standard Definition:

It is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.

The "Black Box" Concept

Think of a car. You interact with the steering wheel and pedals. You don't need to touch the fuel injectors or the timing belt directly. Those internal parts are "encapsulated" away from the driver for safety and simplicity.

2. The Four Motivations

1. Data Hiding

Internal details are hidden from the user. Users only see what they **need** to see. This prevents "code pollution."

2. Increased Security

By making variables private, we prevent external code from setting values that make no sense (like setting an age to -500).

3. Modifiability

You can change the internal data type of a variable (e.g., from `int` to `double`) without breaking the code that uses the object, as long as the public methods stay the same.

4. Debugging Power

If a variable's value is wrong, you only have to check the class's methods to find the bug, rather than searching the entire 10,000-line program.

3. The Recipe for Protection

To implement encapsulation in C++, follow these two mandatory steps:

1. Declare class variables as **private**.
2. Provide **public** functions (Getters and Setters) to access and modify those variables.

The Visibility Wall:

```
class BankAccount {  
private:  
    double balance; // Hidden from main()  
  
public:  
    // Controlled access points  
    void setBalance(double b);  
    double getBalance();  
};
```

4. Setters: The Gatekeepers

A **Setter** is a method used to update a private variable. Its power lies in **Validation**.

```
void setAge(int a) {  
    if (a ≥ 0 && a ≤ 120) {  
        age = a;  
    } else {  
        cout << "Error: Invalid age provided!";  
    }  
}
```

Without encapsulation, `obj.age = -10` would happen silently and crash your logic later. With a setter, the object protects itself.

5. Getters: The Reporters

A **Getter** allows the outside world to "see" a value without being able to change it directly. It usually returns the value or a formatted version of it.

```
string getScore() {  
    return "Score: " + to_string(points);  
}
```

Read-only Data:

If you create a class with a Getter but **no Setter**, you effectively create a variable that can be seen but never changed—crucial for sensitive IDs or serial numbers.

6. CodeHive Implementation

```
class CodeHive {
private:
    string courseName;
    int duration;

public:
    void setCourseName(string c) { courseName = c; }
    string getCourseName() { return courseName; }

    void setDuration(int d) {
        if (d > 0) duration = d;
    }
    int getDuration() { return duration; }
};
```

This class is now a "Safe Object." Its internal variables are shielded.

7. Decoupling Logic

Encapsulation creates a clean boundary. In the future, if you decide that duration should be stored in days instead of months, you only change the math **inside** the class.

```
// Internal change (Outside world still calls getDuration())
int getDuration() {
    return durationInDays / 30;
}
```

The "Users" of your class don't even need to know you changed the internal storage logic. This is the definition of flexible code.

8. Encapsulation vs. Abstraction

These two are often confused. Here is the distinction:

Encapsulation (Hiding Data)

Concerned with **how** a data member is accessed and protected. Focusing on security and wrapping.

Abstraction (Hiding Complexity)

Concerned with **what** the object does rather than how. Focusing on providing a simple interface (like a remote control).

Keyphrase: Encapsulation is a sub-tool used to enforce Abstraction.

9. The Encapsulation Protocol

1. Private by Default: Always start with `private` access specifiers. Move to `protected` or `public` only when absolutely necessary.

2. Naming Conventions: Use `getVariableName()` and `setVariableName()` for consistency.

3. Const Getters: Mark getters as `const` to prove they don't modify the object.

```
int getVal() const { return val; }
```

10. Summary & Skill Challenge

- **Encapsulation:** Wrapping data + behavior.
- **Data Hiding:** Private as a fortress.
- **Getters/Setters:** The secure airlocks.
- **Validation:** Keeping data clean.

Challenge Task:

The Temperature Controller: Write a class `Thermometer` with a private `double celsius`. Create a setter that only allows values above absolute zero (-273.15). Create a getter that returns the temperature in Fahrenheit.

ENCLOSURE COMPLETE: DATA INTEGRITY ACHIEVED